Hanoi Towers Illegal State Algorithms Performance Analysis

Luke Pepin

2025-12-01

```r
# Read data from CSV
data <- read.csv("results.csv")

# Convert to factors with readable labels
data$stack_algorithm <- factor(data$stack_algorithm,
                               levels = c("dig_out", "a_star"),
                               labels = c("Dig-Out", "A*"))
data$ground_algorithm <- factor(data$ground_algorithm,
                                levels = c("greedy", "patient"),
                                labels = c("Greedy", "Patient"))
data$corruption_rate <- factor(data$corruption_rate,
                               levels = c(0.05, 0.1),
                               labels = c("5%", "10%"))

# Rename for clarity in analysis
data$stack <- data$stack_algorithm
data$ground <- data$ground_algorithm
data$corruption <- data$corruption_rate
data$value <- data$total_moves

str(data)
```

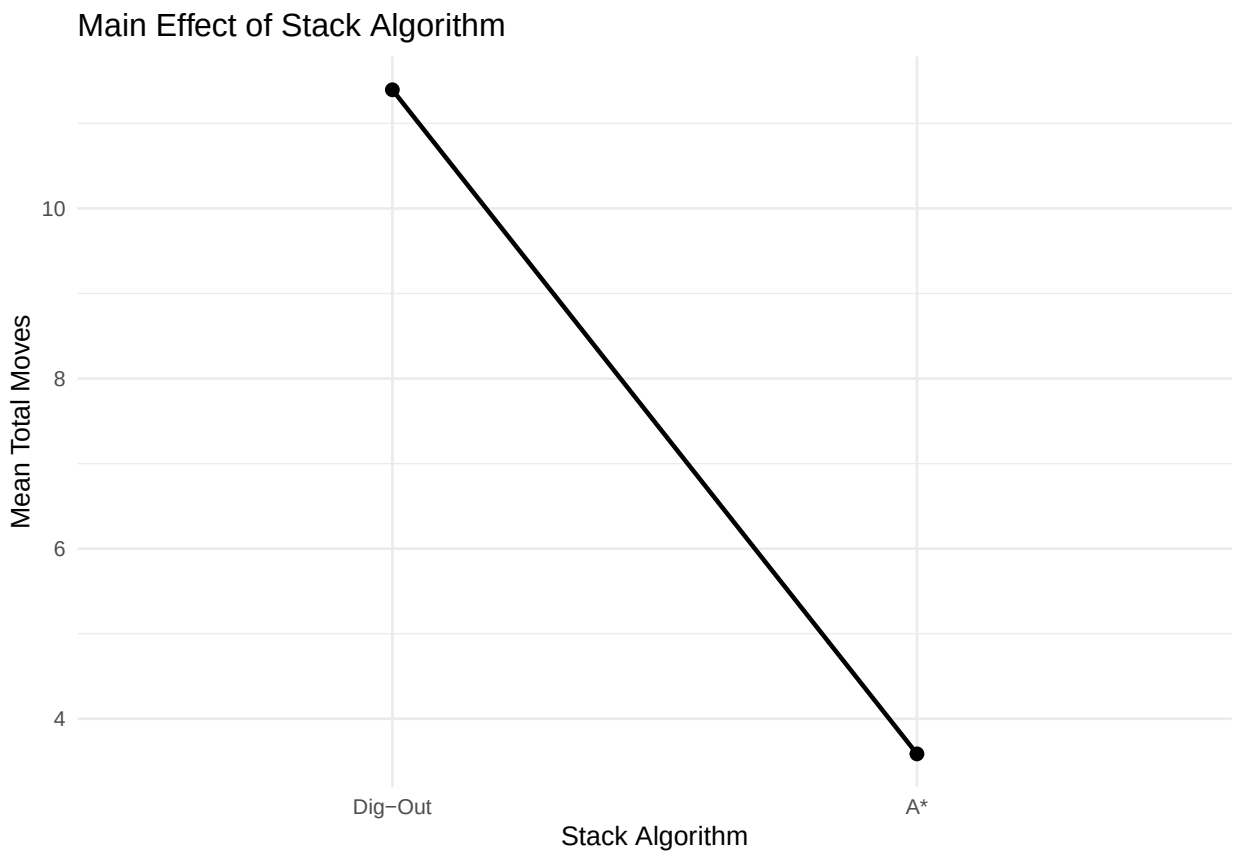
```
## 'data.frame':    800 obs. of  15 variables:
##  $ trial_id               : chr  "C1_T1" "C1_T2" "C1_T3" "C1_T4" ...
##  $ condition_id           : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ stack_algorithm        : Factor w/ 2 levels "Dig-Out","A*": 1 1 1 1 1 1 1 1 1 1 ...
##  $ ground_algorithm       : Factor w/ 2 levels "Greedy","Patient": 1 1 1 1 1 1 1 1 1 1 ...
##  $ corruption_rate        : Factor w/ 2 levels "5%","10%": 1 1 1 1 1 1 1 1 1 1 ...
##  $ seed                   : int  100 200 300 400 500 600 700 800 900 1000 ...
##  $ total_moves            : int  9 4 19 5 5 15 25 5 13 28 ...
##  $ num_regenerations      : int  2 1 1 1 1 4 2 1 1 1 ...
##  $ num_corruptions_occurred: int  1 0 0 0 0 3 2 0 0 0 ...
##  $ final_state_valid      : chr  "True" "True" "True" "True" ...
##  $ timeout                : chr  "False" "False" "False" "False" ...
##  $ stack                  : Factor w/ 2 levels "Dig-Out","A*": 1 1 1 1 1 1 1 1 1 1 ...
##  $ ground                 : Factor w/ 2 levels "Greedy","Patient": 1 1 1 1 1 1 1 1 1 1 ...
##  $ corruption             : Factor w/ 2 levels "5%","10%": 1 1 1 1 1 1 1 1 1 1 ...
##  $ value                  : int  9 4 19 5 5 15 25 5 13 28 ...
```

```r
head(data)
```

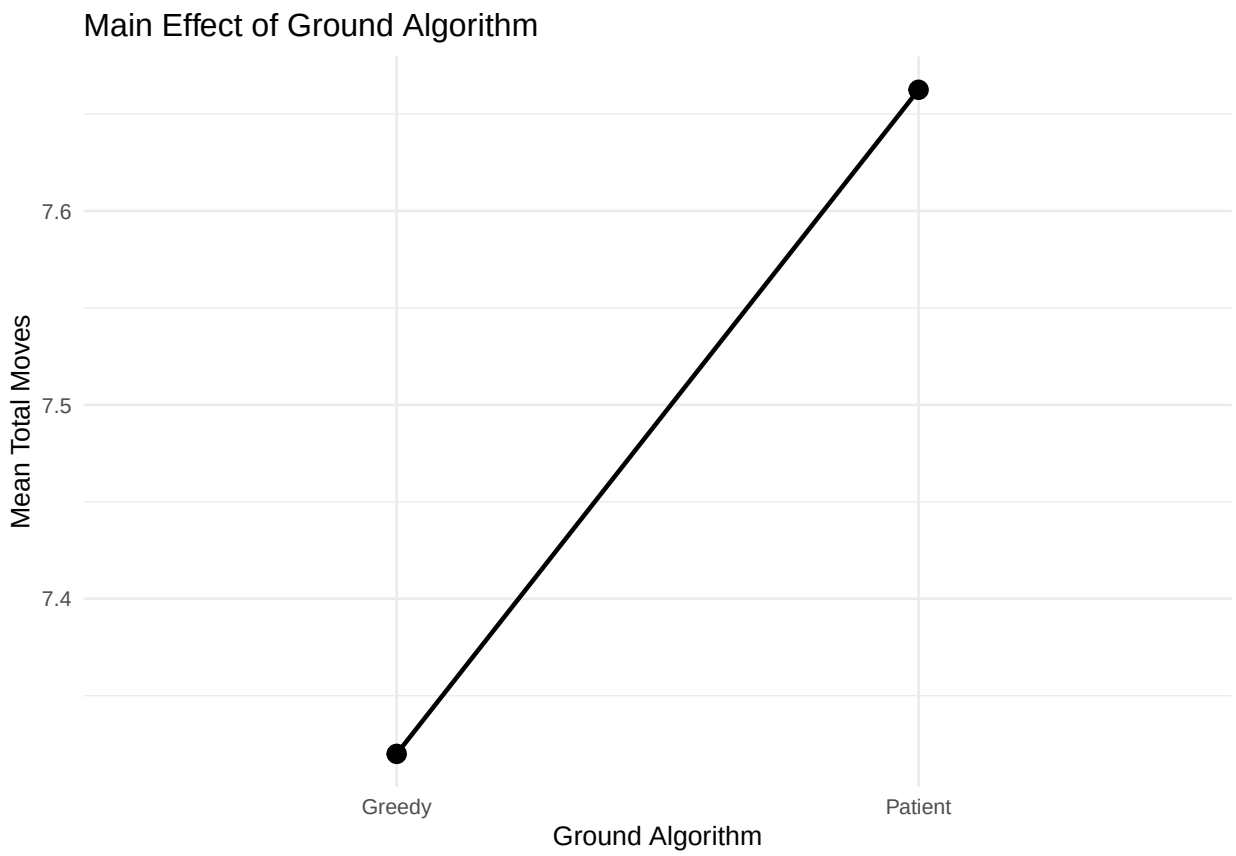
```
##   trial_id condition_id stack_algorithm ground_algorithm corruption_rate seed
```

```
## 1    C1_T1            1         Dig-Out           Greedy               5%  100
## 2    C1_T2            1         Dig-Out           Greedy               5%  200
## 3    C1_T3            1         Dig-Out           Greedy               5%  300
## 4    C1_T4            1         Dig-Out           Greedy               5%  400
## 5    C1_T5            1         Dig-Out           Greedy               5%  500
## 6    C1_T6            1         Dig-Out           Greedy               5%  600
##   total_moves num_regenerations num_corruptions_occurred final_state_valid
## 1           9                 2                         1              True
## 2           4                 1                         0              True
## 3          19                 1                         0              True
## 4           5                 1                         0              True
## 5           5                 1                         0              True
## 6          15                 4                         3              True
##   timeout   stack ground corruption value
## 1   False Dig-Out Greedy         5%     9
## 2   False Dig-Out Greedy         5%     4
## 3   False Dig-Out Greedy         5%    19
## 4   False Dig-Out Greedy         5%     5
## 5   False Dig-Out Greedy         5%     5
## 6   False Dig-Out Greedy         5%    15
```

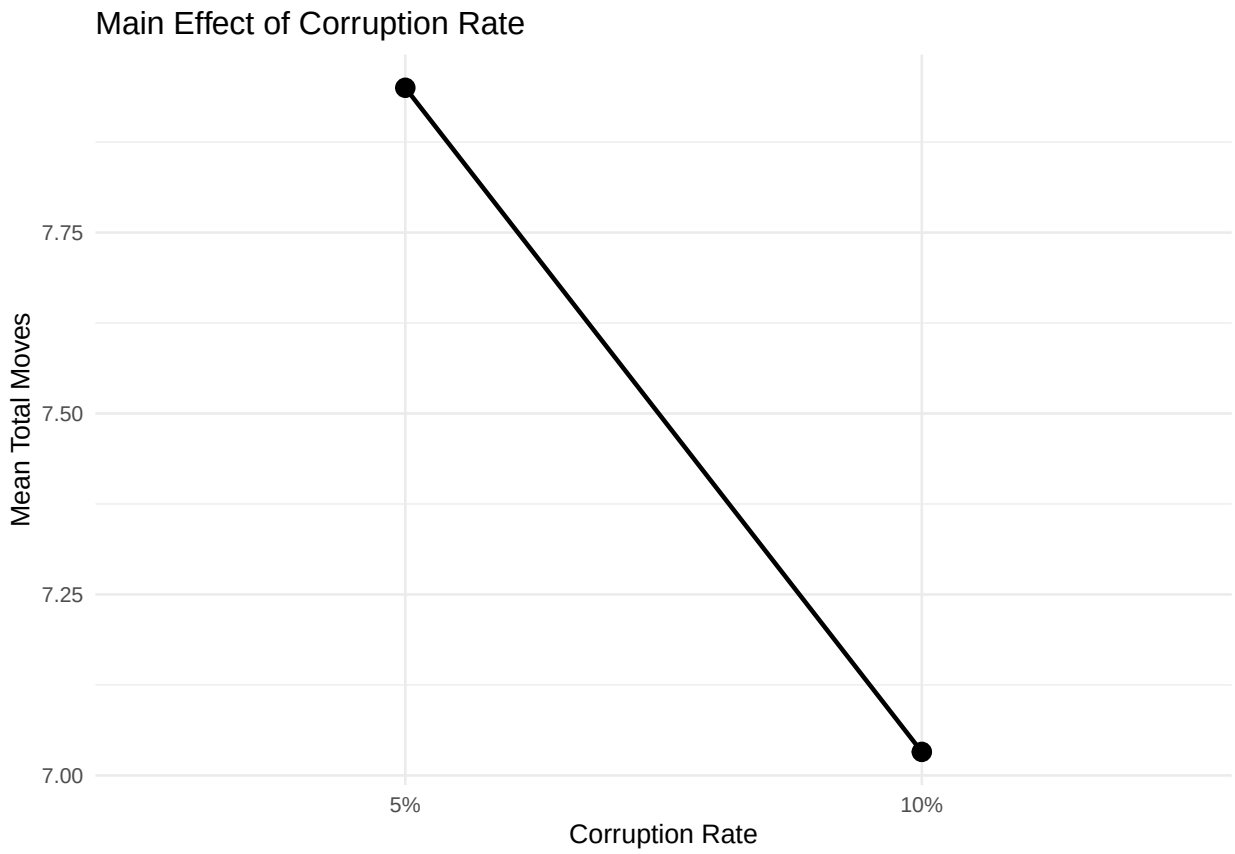
```r
# Main effect: Stack Algorithm
ggplot(data, aes(x = stack, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 2) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Stack Algorithm",
       y = "Mean Total Moves",
       x = "Stack Algorithm") +
  theme_minimal(base_size = 10)
```




```
# Main effect: Ground Algorithm
ggplot(data, aes(x = ground, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Ground Algorithm",
       y = "Mean Total Moves",
       x = "Ground Algorithm") +
  theme_minimal(base_size = 10)
```




```
# Main effect: Corruption Rate
ggplot(data, aes(x = corruption, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), linewidth = 0.8) +
  labs(title = "Main Effect of Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate") +
  theme_minimal(base_size = 10)
```

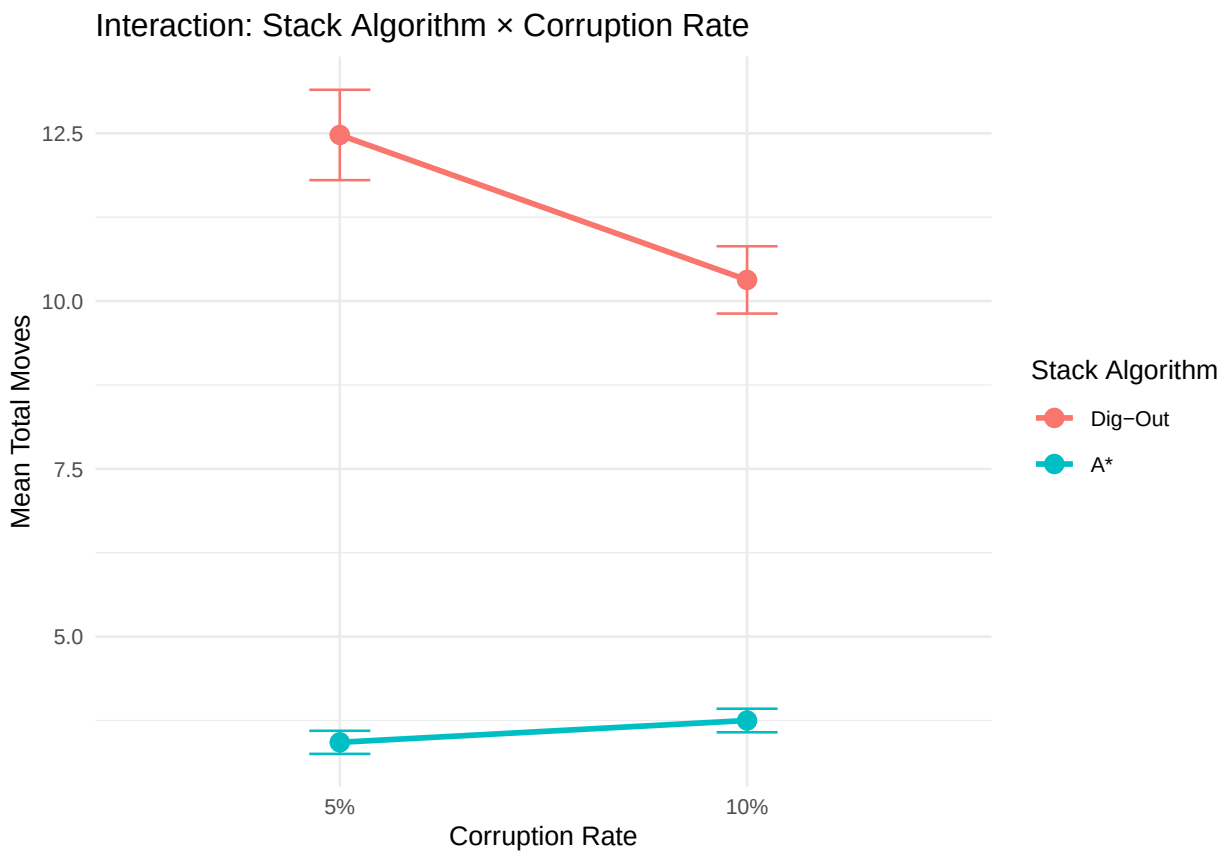



```r
# Calculate group means for interaction plots
group_means <- data %>%
  group_by(stack, ground, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

print(group_means)
```

```
## # A tibble: 8 x 7
##   stack   ground  corruption mean_value    sd     n    se
##   <fct>   <fct>   <fct>           <dbl> <dbl> <int> <dbl>
## 1 Dig-Out Greedy  5%              11.3   7.70   100 0.770
## 2 Dig-Out Greedy  10%             10.9   7.33   100 0.733
## 3 Dig-Out Patient 5%              13.6  11.0    100 1.10
## 4 Dig-Out Patient 10%              9.74  6.83   100 0.683
## 5 A*      Greedy  5%               3.46  2.50   100 0.250
## 6 A*      Greedy  10%              3.6   2.53   100 0.253
## 7 A*      Patient 5%               3.39  2.38   100 0.238
## 8 A*      Patient 10%              3.9   2.44   100 0.244
```

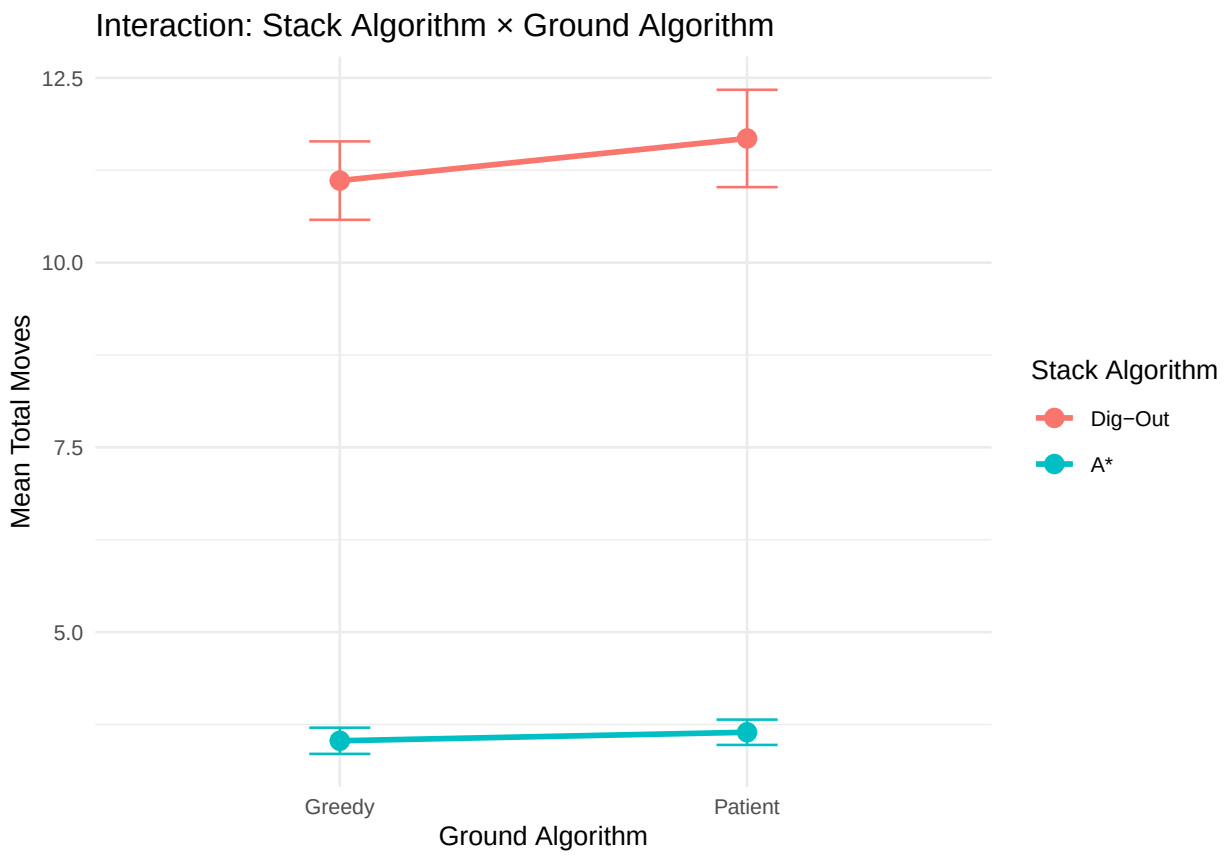
```r
# Two-way interaction plots
# 1. Stack × Corruption
group_means_sc <- data %>%
  group_by(stack, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

ggplot(group_means_sc, aes(x = corruption, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Stack Algorithm × Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate",
       color = "Stack Algorithm") +
  theme_minimal(base_size = 10)
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

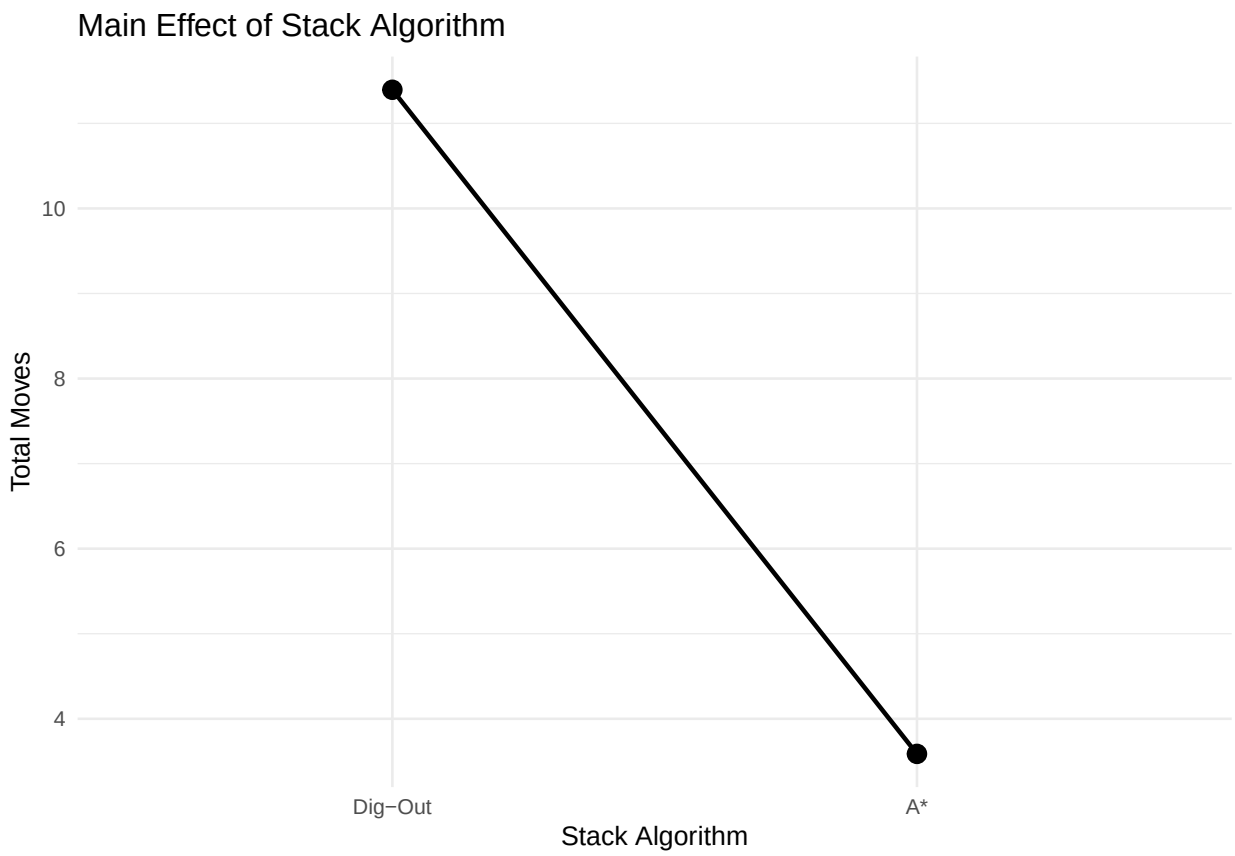



```r
# 2. Ground × Corruption
group_means_gc <- data %>%
  group_by(ground, corruption) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

ggplot(group_means_gc, aes(x = corruption, y = mean_value, color = ground, group = ground)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Ground Algorithm × Corruption Rate",
       y = "Mean Total Moves",
       x = "Corruption Rate",
       color = "Ground Algorithm") +
  theme_minimal(base_size = 10)
```

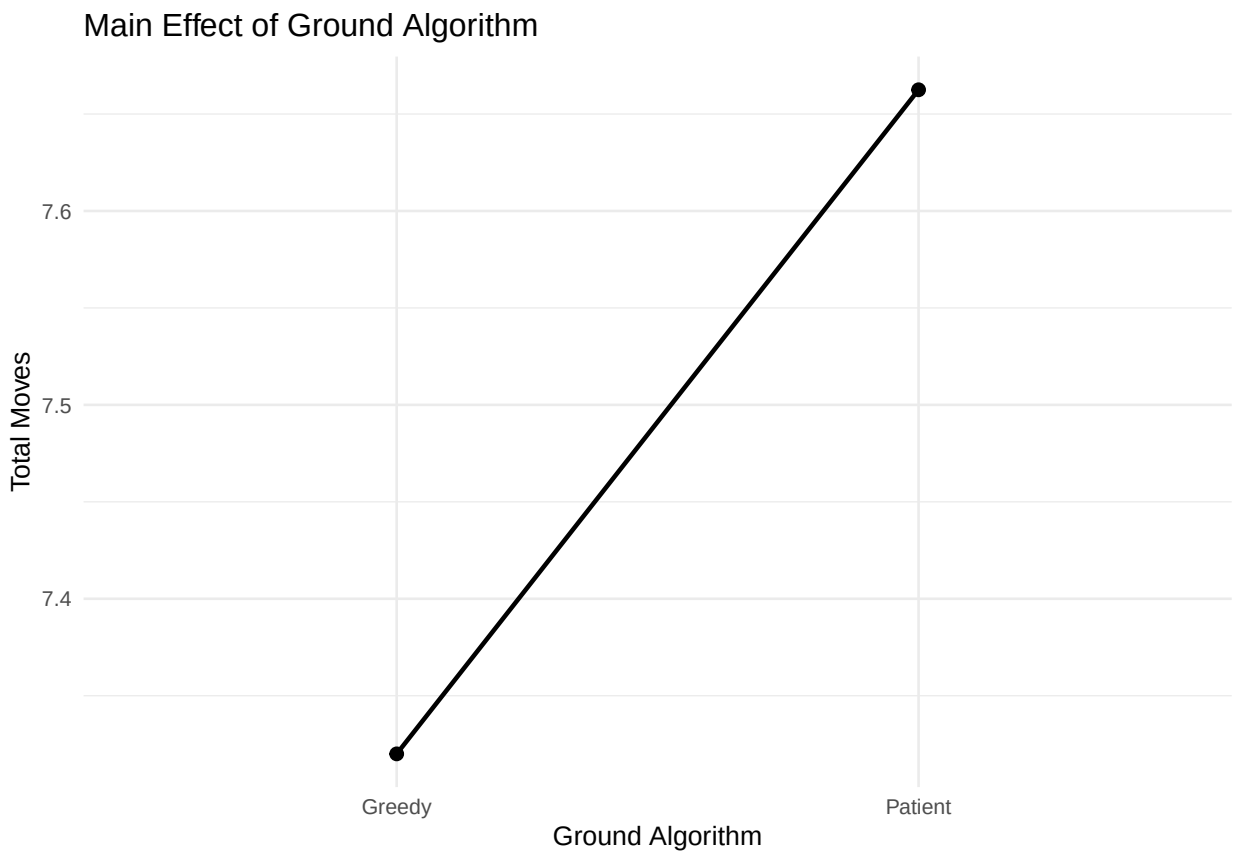

```r
# 3. Stack × Ground
group_means_sg <- data %>%
  group_by(stack, ground) %>%
  summarise(
    mean_value = mean(value),
    sd = sd(value),
    n = n(),
    se = sd / sqrt(n),
    .groups = 'drop'
  )

ggplot(group_means_sg, aes(x = ground, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  labs(title = "Interaction: Stack Algorithm × Ground Algorithm",
       y = "Mean Total Moves",
       x = "Ground Algorithm",
       color = "Stack Algorithm") +
  theme_minimal(base_size = 10)
```



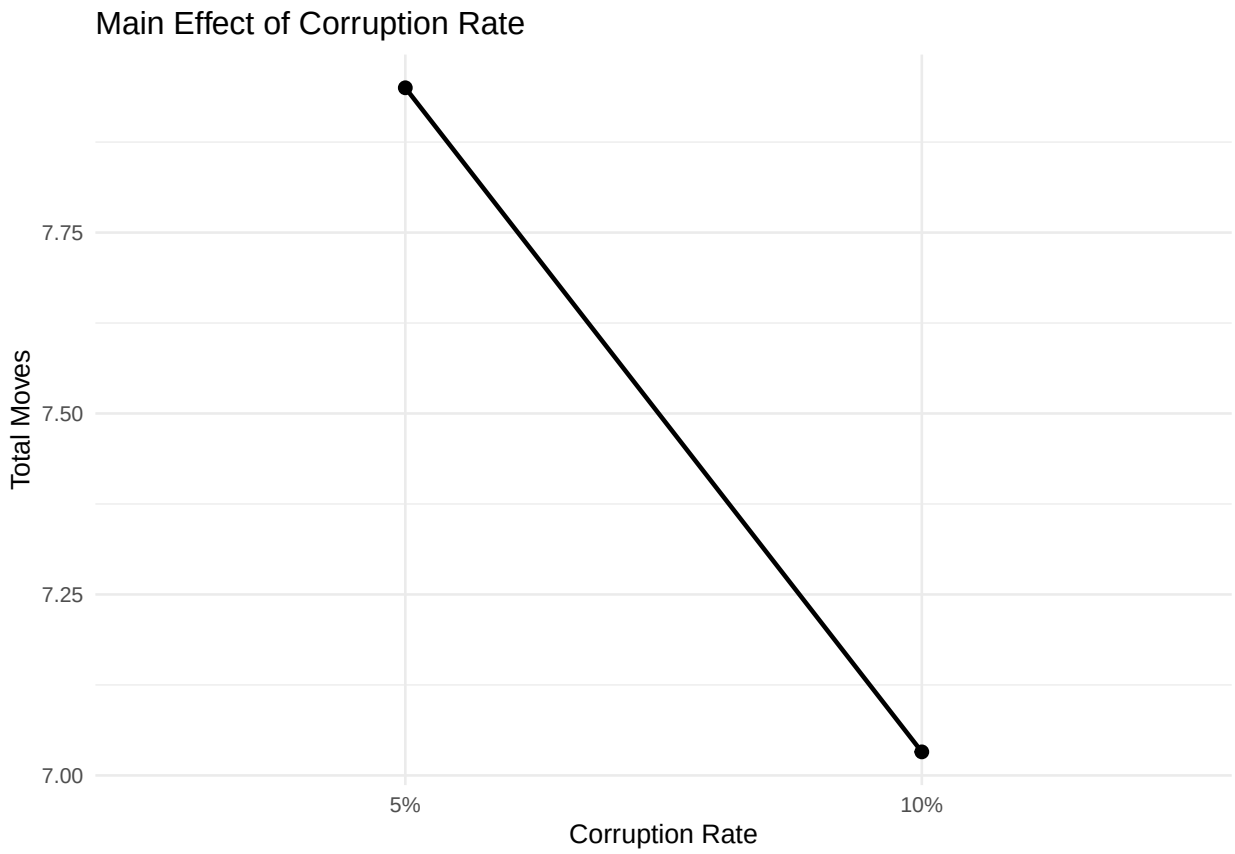

```r
# Create visualization of main effects
# 1. Stack Algorithm effect
p1 <- ggplot(data, aes(x = stack, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), size = 0.8) +
  labs(title = "Main Effect of Stack Algorithm",
       y = "Total Moves",
       x = "Stack Algorithm") +
  theme_minimal(base_size = 10)
print(p1)
```



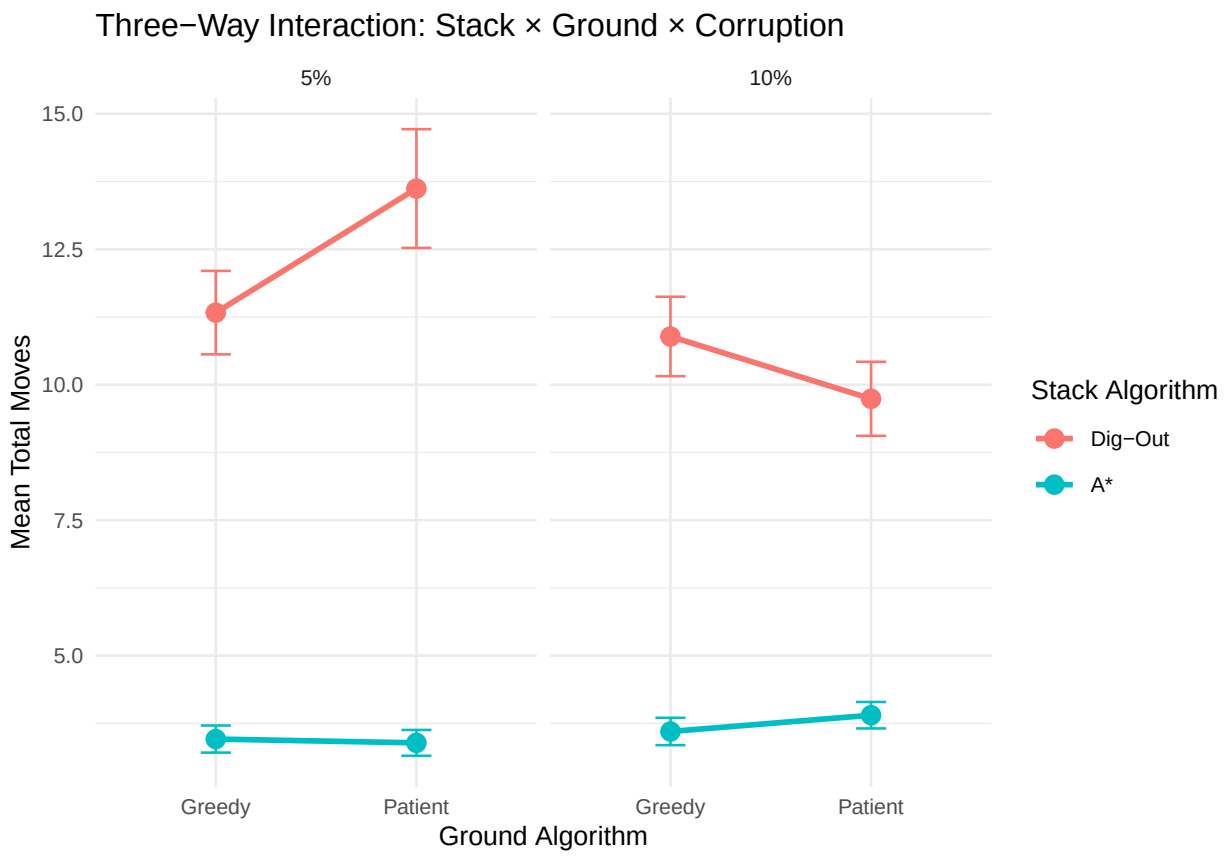

```r
# 2. Ground Algorithm effect
p2 <- ggplot(data, aes(x = ground, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 2) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), size = 0.8) +
  labs(title = "Main Effect of Ground Algorithm",
       y = "Total Moves",
       x = "Ground Algorithm") +
  theme_minimal(base_size = 10)
print(p2)
```




```r
# 3. Corruption Rate effect
p3 <- ggplot(data, aes(x = corruption, y = value)) +
  stat_summary(fun = mean, geom = "point", size = 2) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), size = 0.8) +
  labs(title = "Main Effect of Corruption Rate",
       y = "Total Moves",
       x = "Corruption Rate") +
  theme_minimal(base_size = 10)
print(p3)
```



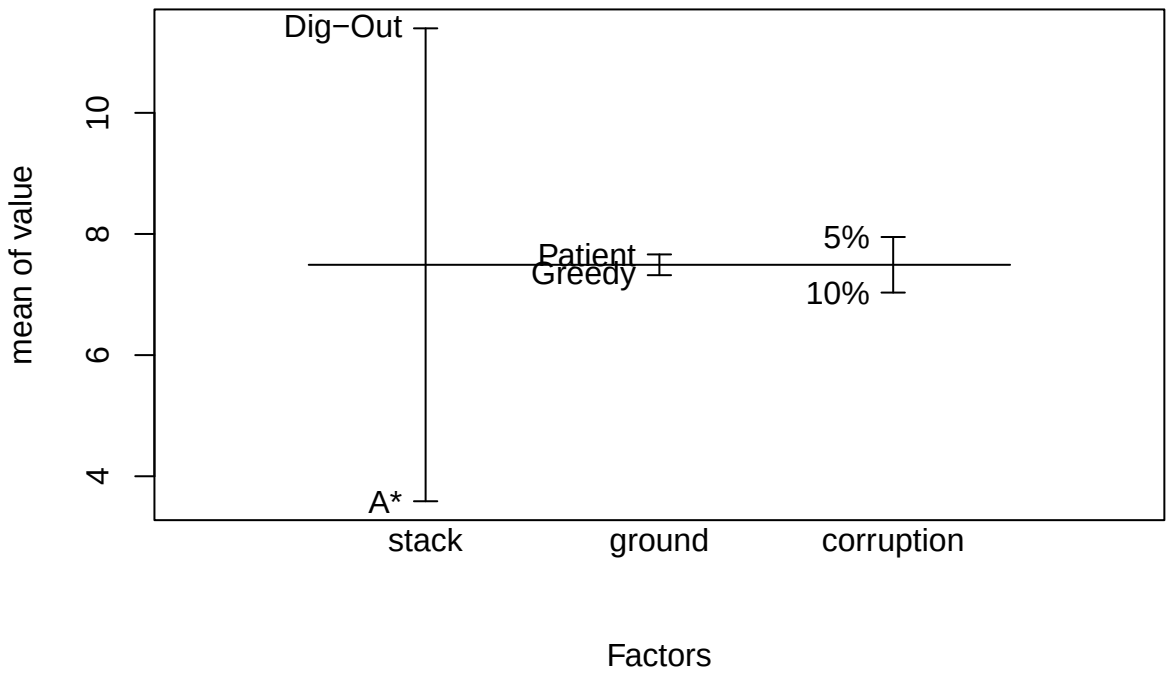

```r
# 4. Three-way interaction plot (Stack × Ground faceted by Corruption)
p4 <- ggplot(group_means, aes(x = ground, y = mean_value, color = stack, group = stack)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_value - se, ymax = mean_value + se), width = 0.15) +
  facet_wrap(~corruption) +
  labs(
    title = "Three-Way Interaction: Stack × Ground × Corruption",
    x = "Ground Algorithm",
    y = "Mean Total Moves",
    color = "Stack Algorithm"
  ) +
  theme_minimal(base_size = 10)
print(p4)
```




```r
# Fit three-way ANOVA model with all interactions
# Model: Total Moves ~ Stack * Ground * Corruption
model <- aov(value ~ stack * ground * corruption, data = data)

# Display ANOVA table with significance tests
summary(model)
```

```
##                          Df Sum Sq Mean Sq F value  Pr(>F)
## stack                     1  12191   12191 320.610 < 2e-16 ***
## ground                    1     23      23   0.617 0.43240
## corruption                1    168     168   4.428 0.03568 *
## stack:ground              1     10      10   0.272 0.60199
## stack:corruption          1    309     309   8.120 0.00449 **
## ground:corruption         1    118     118   3.098 0.07876 .
## stack:ground:corruption   1    181     181   4.772 0.02922 *
## Residuals               792  30116      38
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Store results and create effects plot
anova_sum <- summary(model)[[1]]
plot.design(value ~ stack + ground + corruption, data = data)
```




```r
# Sample Size Calculation
k <- 8  # Number of groups (2 stack × 2 ground × 2 corruption)
f <- 0.25  # Small-medium effect size for algorithm comparisons
pwr_result <- pwr.anova.test(k = k, f = f, sig.level = 0.05, power = 0.9)
cat("Required sample size per group:", ceiling(pwr_result$n), "\n")
```

```
## Required sample size per group: 38
```

```r
# Calculate statistical power from the experiment
# Define parameters
k <- 8 # total number of groups (2 × 2 × 2)
n_per_group <- nrow(data) / k # samples per group
f <- 0.25 # small-medium effect size for algorithm performance
# Calculate power
sample_calc <- pwr.anova.test(k = k, n = n_per_group, f = f, sig.level = 0.05)$power
# Print results
cat("Actual power achieved:", round(sample_calc, 4), "\n")
```

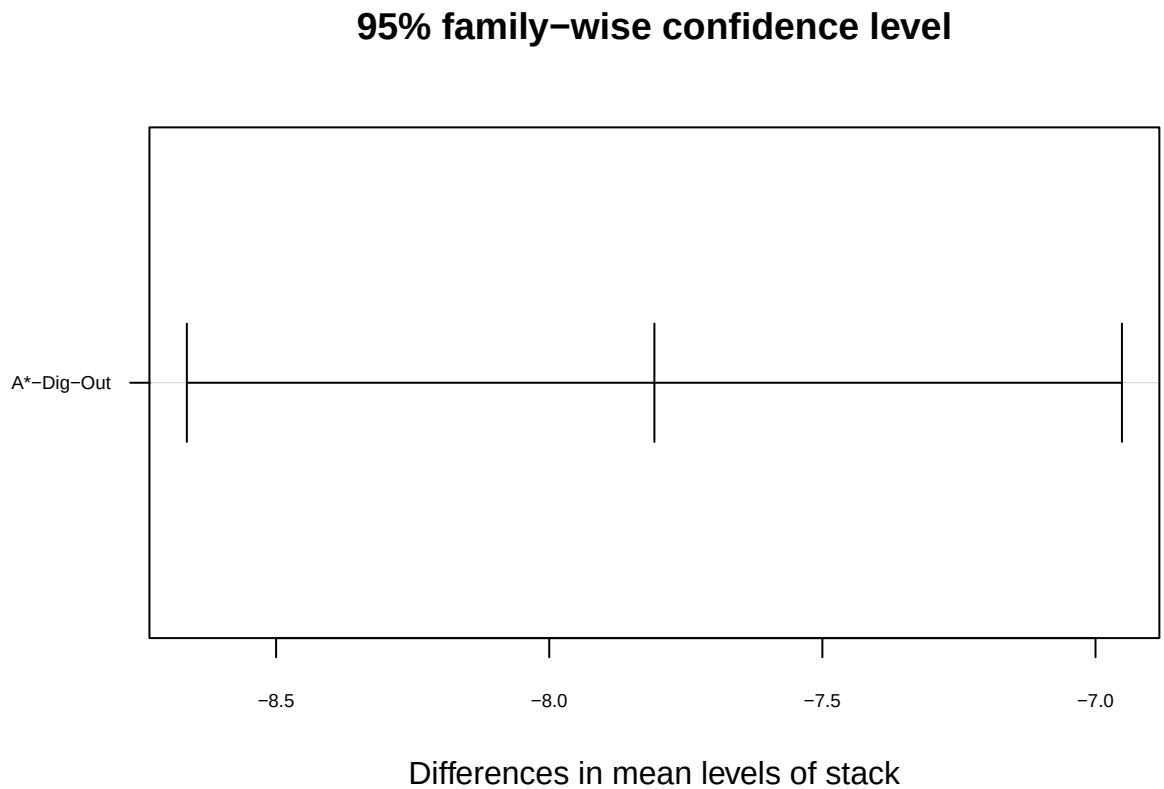
```
## Actual power achieved: 0.9999
```

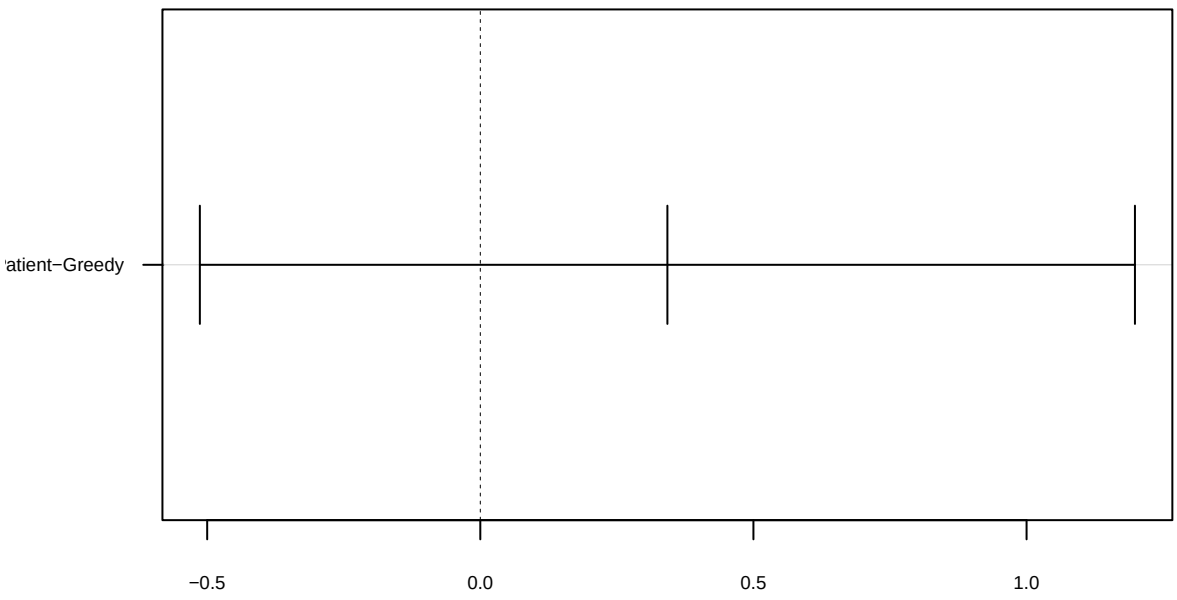
```r
# Tukey's HSD Test
tukey_result <- TukeyHSD(model)
print(tukey_result)
```

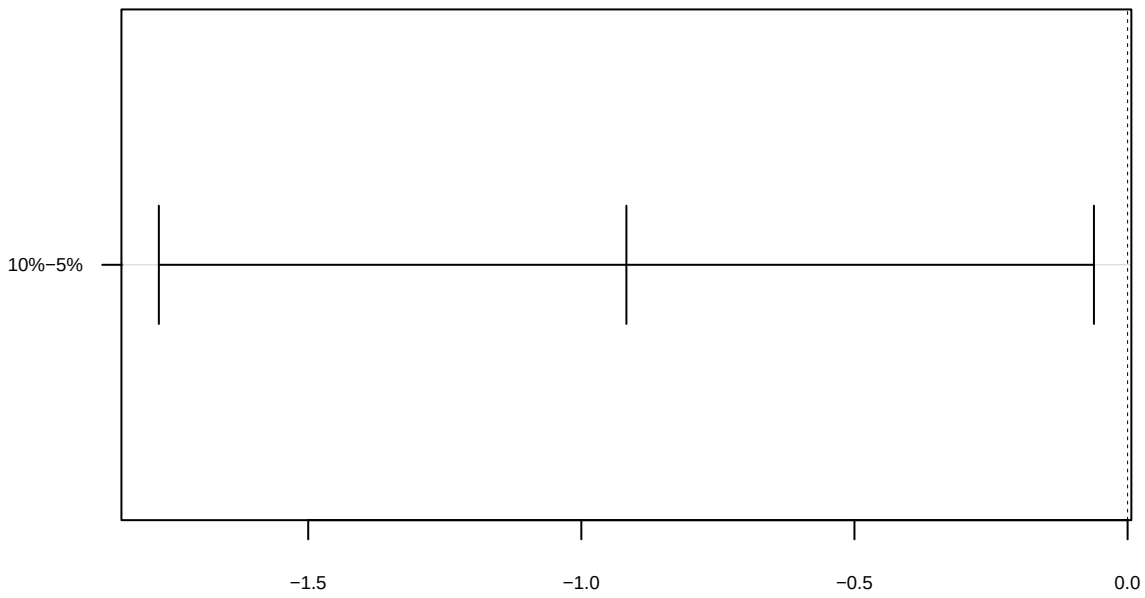
```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##
## Fit: aov(formula = value ~ stack * ground * corruption, data = data)
##
## $stack
##               diff       lwr       upr p adj
## A*-Dig-Out -7.8075 -8.663425 -6.951575     0
##
## $ground
##                   diff       lwr      upr    p adj
## Patient-Greedy 0.3425 -0.513425 1.198425 0.432405
##
## $corruption
##           diff       lwr         upr     p adj
## 10%-5% -0.9175 -1.773425 -0.06157501 0.0356775
##
## $`stack:ground`
##                                    diff      lwr      upr     p adj
## A*:Greedy-Dig-Out:Greedy       -7.580 -9.16756 -5.99244 0.0000000
## Dig-Out:Patient-Dig-Out:Greedy  0.570 -1.01756  2.15756 0.7918235
## A*:Patient-Dig-Out:Greedy      -7.465 -9.05256 -5.87744 0.0000000
## Dig-Out:Patient-A*:Greedy       8.150  6.56244  9.73756 0.0000000
## A*:Patient-A*:Greedy            0.115 -1.47256  1.70256 0.9976985
## A*:Patient-Dig-Out:Patient     -8.035 -9.62256 -6.44744 0.0000000
##
## $`stack:corruption`
##                             diff       lwr        upr     p adj
## A*:5%-Dig-Out:5%       -9.050 -10.63756 -7.4624396 0.0000000
## Dig-Out:10%-Dig-Out:5% -2.160  -3.74756 -0.5724396 0.0027327
## A*:10%-Dig-Out:5%      -8.725 -10.31256 -7.1374396 0.0000000
## Dig-Out:10%-A*:5%       6.890   5.30244  8.4775604 0.0000000
## A*:10%-A*:5%            0.325  -1.26256  1.9125604 0.9525367
## A*:10%-Dig-Out:10%     -6.565  -8.15256 -4.9774396 0.0000000
##
## $`ground:corruption`
##                              diff        lwr         upr     p adj
## Patient:5%-Greedy:5%    1.110 -0.4775604  2.69756039 0.2740371
## Greedy:10%-Greedy:5%   -0.150 -1.7375604  1.43756039 0.9949391
## Patient:10%-Greedy:5%  -0.575 -2.1625604  1.01256039 0.7874198
## Greedy:10%-Patient:5%  -1.260 -2.8475604  0.32756039 0.1730897
## Patient:10%-Patient:5% -1.685 -3.2725604 -0.09743961 0.0325144
## Patient:10%-Greedy:10% -0.425 -2.0125604  1.16256039 0.9012298
##
## $`stack:ground:corruption`
##                                                diff         lwr         upr     p adj
## A*:Greedy:5%-Dig-Out:Greedy:5%                -7.87 -10.5202641 -5.21973588 0.0000000
## Dig-Out:Patient:5%-Dig-Out:Greedy:5%           2.29  -0.3602641  4.94026412 0.1481288
## A*:Patient:5%-Dig-Out:Greedy:5%               -7.94 -10.5902641 -5.28973588 0.0000000
## Dig-Out:Greedy:10%-Dig-Out:Greedy:5%          -0.44  -3.0902641  2.21026412 0.9996403
## A*:Greedy:10%-Dig-Out:Greedy:5%               -7.73 -10.3802641 -5.07973588 0.0000000
## Dig-Out:Patient:10%-Dig-Out:Greedy:5%         -1.59  -4.2402641  1.06026412 0.6043445
## A*:Patient:10%-Dig-Out:Greedy:5%              -7.43 -10.0802641 -4.77973588 0.0000000
## Dig-Out:Patient:5%-A*:Greedy:5%               10.16   7.5097359 12.81026412 0.0000000
```

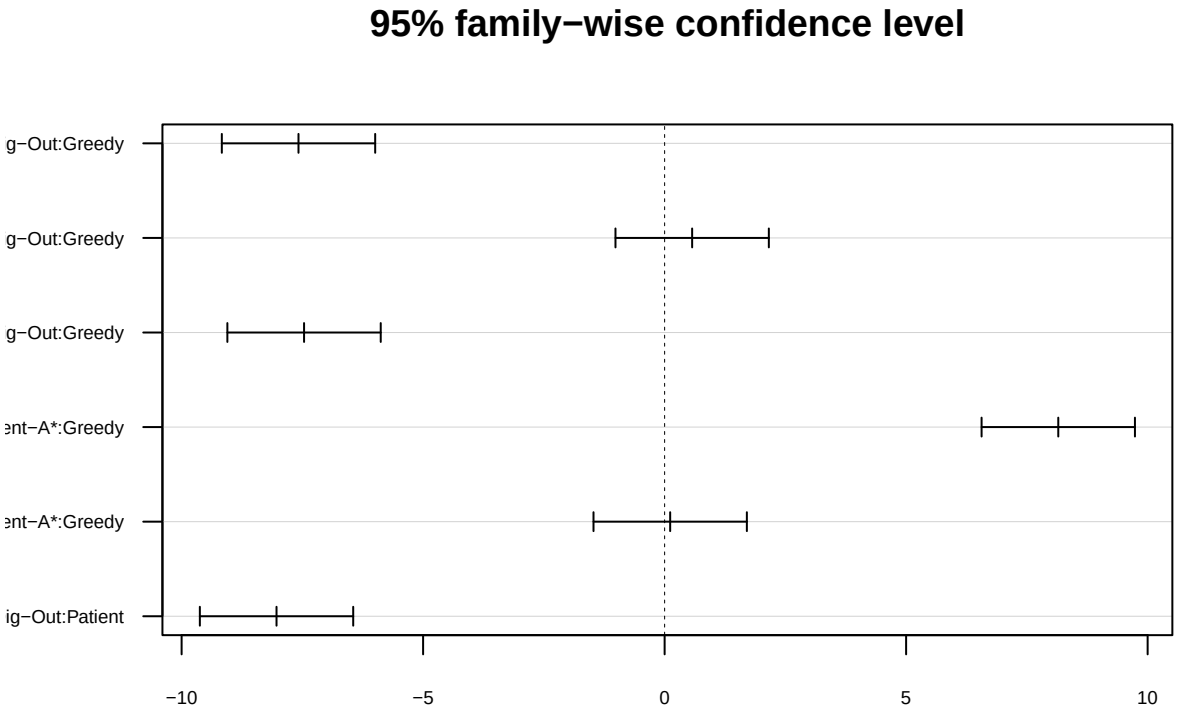
```
## A*:Patient:5%-A*:Greedy:5%                -0.07  -2.7202641  2.58026412 1.0000000
## Dig-Out:Greedy:10%-A*:Greedy:5%            7.43   4.7797359 10.08026412 0.0000000
## A*:Greedy:10%-A*:Greedy:5%                 0.14  -2.5102641  2.79026412 0.9999999
## Dig-Out:Patient:10%-A*:Greedy:5%           6.28   3.6297359  8.93026412 0.0000000
## A*:Patient:10%-A*:Greedy:5%                0.44  -2.2102641  3.09026412 0.9996403
## A*:Patient:5%-Dig-Out:Patient:5%         -10.23 -12.8802641 -7.57973588 0.0000000
## Dig-Out:Greedy:10%-Dig-Out:Patient:5%    -2.73  -5.3802641 -0.07973588 0.0381435
## A*:Greedy:10%-Dig-Out:Patient:5%         -10.02 -12.6702641 -7.36973588 0.0000000
## Dig-Out:Patient:10%-Dig-Out:Patient:5%   -3.88  -6.5302641 -1.22973588 0.0002633
## A*:Patient:10%-Dig-Out:Patient:5%        -9.72 -12.3702641 -7.06973588 0.0000000
## Dig-Out:Greedy:10%-A*:Patient:5%           7.50   4.8497359 10.15026412 0.0000000
## A*:Greedy:10%-A*:Patient:5%                0.21  -2.4402641  2.86026412 0.9999977
## Dig-Out:Patient:10%-A*:Patient:5%          6.35   3.6997359  9.00026412 0.0000000
## A*:Patient:10%-A*:Patient:5%               0.51  -2.1402641  3.16026412 0.9990506
## A*:Greedy:10%-Dig-Out:Greedy:10%          -7.29  -9.9402641 -4.63973588 0.0000000
## Dig-Out:Patient:10%-Dig-Out:Greedy:10%    -1.15  -3.8002641  1.50026412 0.8917779
## A*:Patient:10%-Dig-Out:Greedy:10%         -6.99  -9.6402641 -4.33973588 0.0000000
## Dig-Out:Patient:10%-A*:Greedy:10%          6.14   3.4897359  8.79026412 0.0000000
## A*:Patient:10%-A*:Greedy:10%               0.30  -2.3502641  2.95026412 0.9999728
## A*:Patient:10%-Dig-Out:Patient:10%        -5.84  -8.4902641 -3.18973588 0.0000000
```

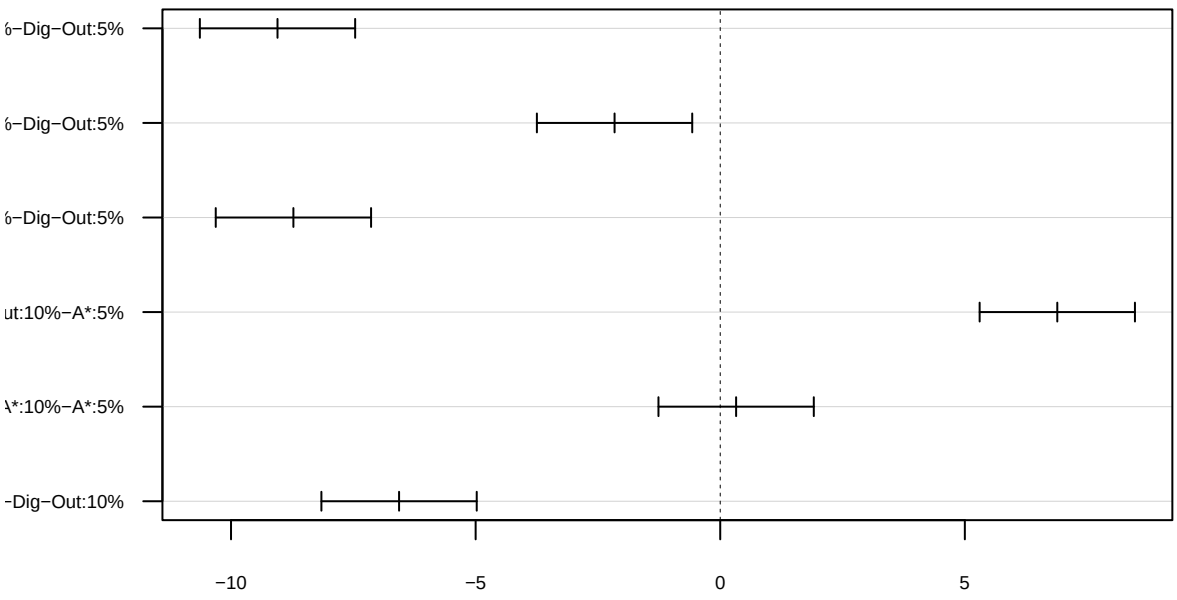
```
plot(tukey_result, las = 1, cex.axis = 0.6)
```

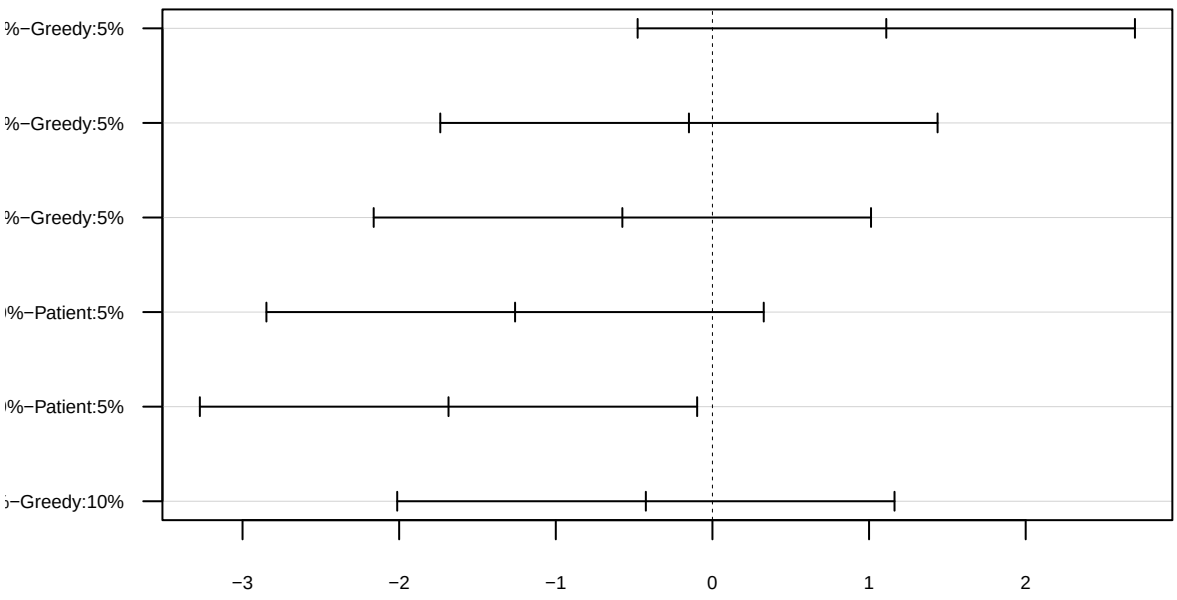


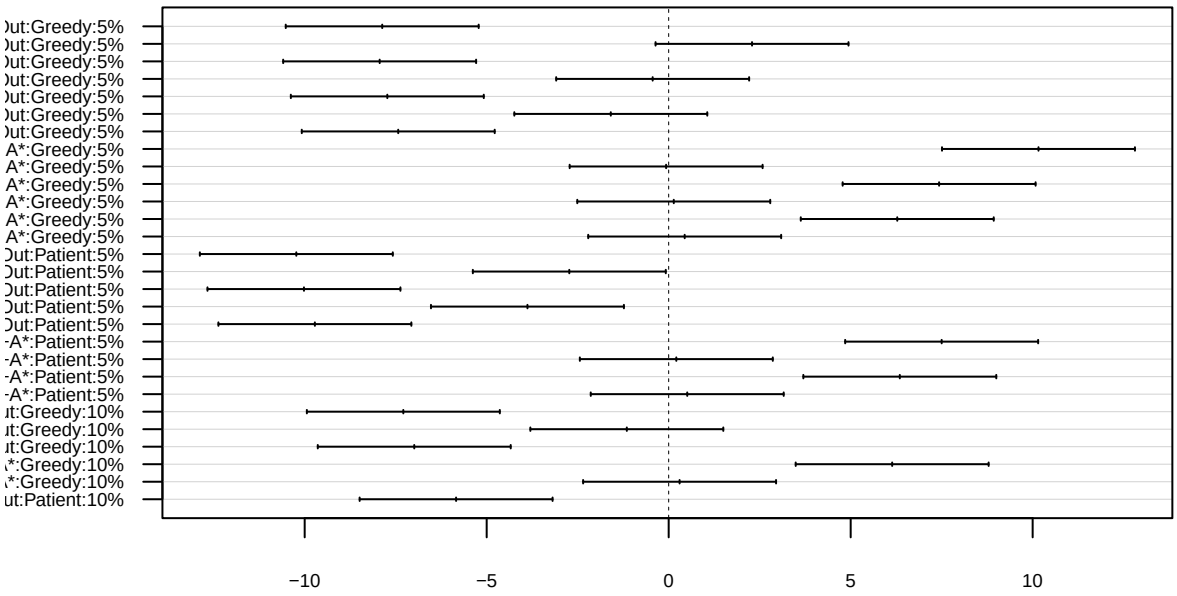

95% family−wise confidence level
atient−Greedy
−0.5
0.0
0.5
1.0
Differences in mean levels of ground


95% family−wise confidence level
10%−5%
−1.5
−1.0
−0.5
0.0
Differences in mean levels of corruption


95% family−wise confidence level
g−Out:Greedy
g−Out:Greedy
g−Out:Greedy
ent−A*:Greedy
ent−A*:Greedy
ig−Out:Patient
−10
−5
0
5
10
Differences in mean levels of stack:ground


95% family−wise confidence level
%−Dig−Out:5%
%−Dig−Out:5%
%−Dig−Out:5%
ut:10%−A*:5%
A*:10%−A*:5%
−Dig−Out:10%
−10
−5
0
5
Differences in mean levels of stack:corruption


95% family−wise confidence level
%−Greedy:5%
%−Greedy:5%
%−Greedy:5%
%−Patient:5%
%−Patient:5%
−Greedy:10%
−3
−2
−1
0
1
2
Differences in mean levels of ground:corruption




```
# Fisher's LSD Test
lsd_resultStack <- LSD.test(model, "stack", p.adj = "none")
lsd_resultGround <- LSD.test(model, "ground", p.adj = "none")
lsd_resultCorr <- LSD.test(model, "corruption", p.adj = "none")
print(lsd_resultStack)
```

```
## $statistics
##    MSerror  Df    Mean       CV  t.value      LSD
##   38.02567 792 7.49125 82.31598 1.962964 0.855925
##
## $parameters
##         test p.ajusted name.t ntr alpha
##   Fisher-LSD      none  stack   2  0.05
##
## $means
##           value      std   r        se      LCL      UCL Min Max Q25 Q50   Q75
## A*       3.5875 2.462214 400 0.3083248  2.98227  4.19273   1  18   2   3  4.00
## Dig-Out 11.3950 8.452668 400 0.3083248 10.78977 12.00023   1  69   5   9 14.25
##
## $comparison
## NULL
##
## $groups
##           value groups
## Dig-Out 11.3950      a
```

```
## A*       3.5875      b
##
## attr(,"class")
## [1] "group"
```

```
print(lsd_resultGround)
```

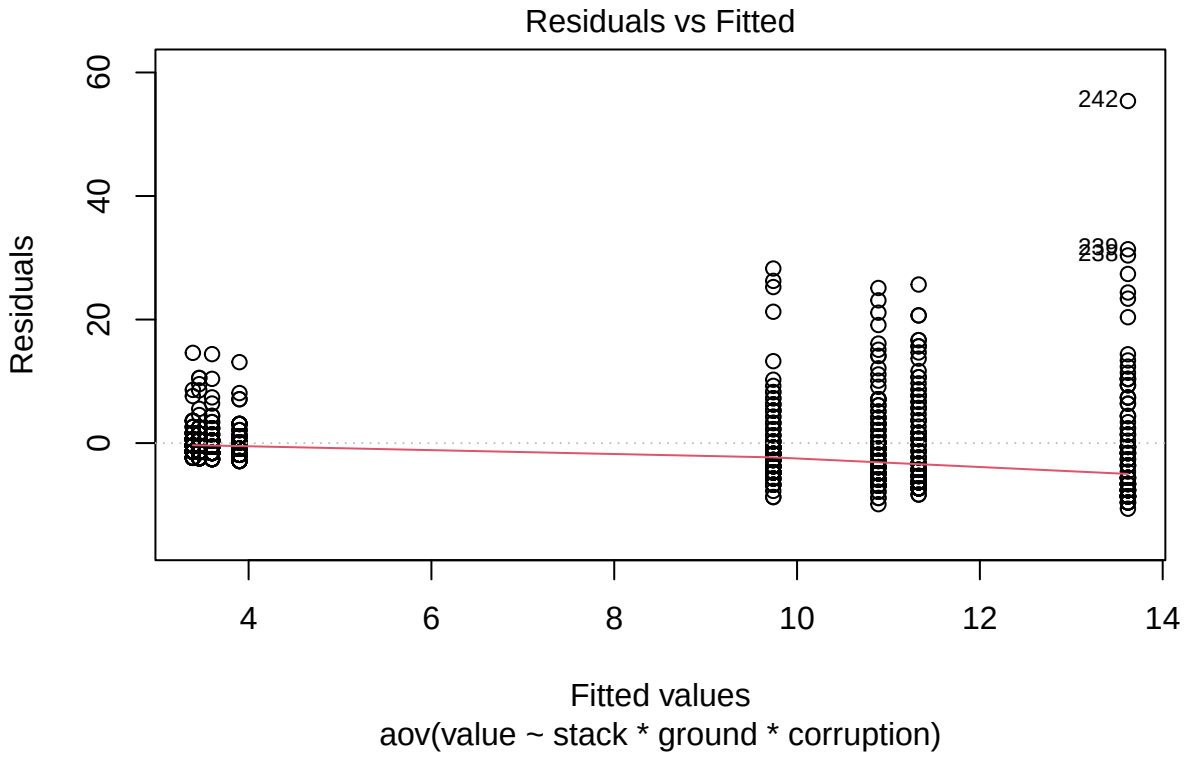
```
## $statistics
##    MSerror  Df    Mean       CV  t.value      LSD
##   38.02567 792 7.49125 82.31598 1.962964 0.855925
##
## $parameters
##         test p.ajusted name.t ntr alpha
##   Fisher-LSD      none ground   2  0.05
##
## $means
##          value      std   r        se     LCL     UCL Min Max Q25 Q50 Q75
## Greedy  7.3200 6.754751 400 0.3083248 6.71477 7.92523   1  37   3   5  10
## Patient 7.6625 7.898072 400 0.3083248 7.05727 8.26773   1  69   3   5   9
##
## $comparison
## NULL
##
## $groups
##          value groups
## Patient 7.6625      a
## Greedy  7.3200      a
##
## attr(,"class")
## [1] "group"
```

```
print(lsd_resultCorr)
```

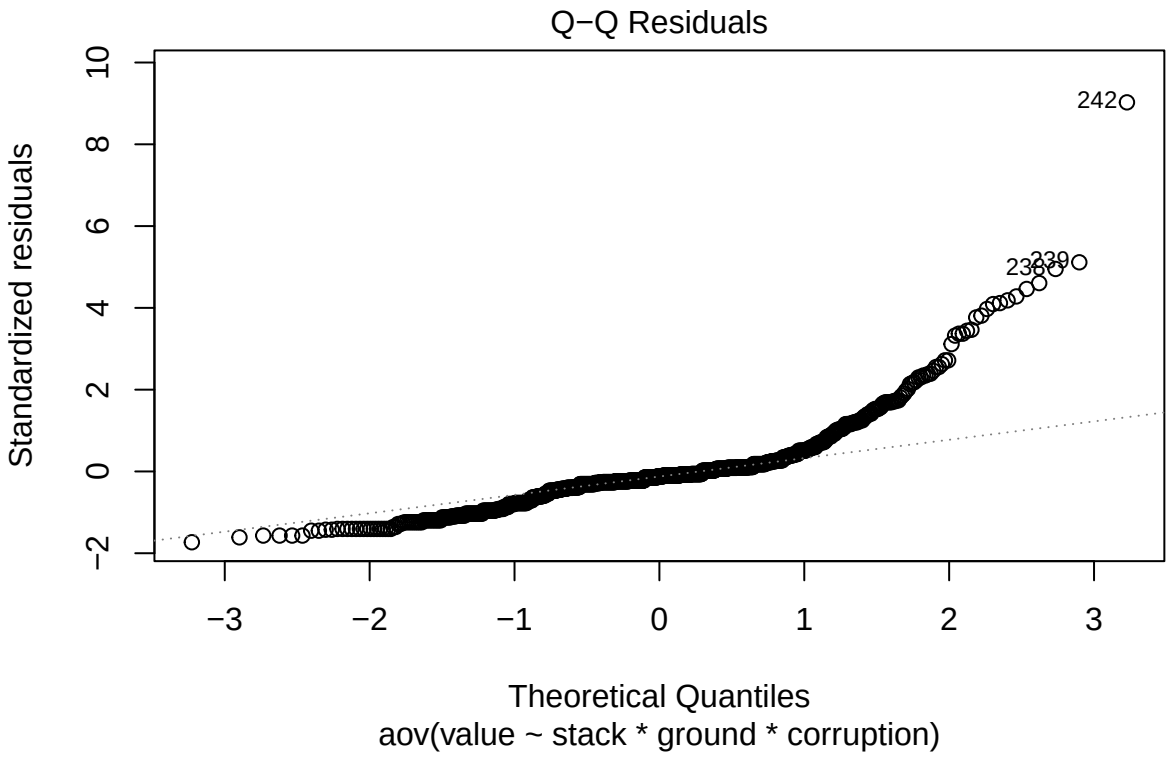
```
## $statistics
##    MSerror  Df    Mean       CV  t.value      LSD
##   38.02567 792 7.49125 82.31598 1.962964 0.855925
##
## $parameters
##         test p.ajusted     name.t ntr alpha
##   Fisher-LSD      none corruption   2  0.05
##
## $means
##      value      std   r        se     LCL     UCL Min Max Q25 Q50   Q75
## 10% 7.0325 6.242505 400 0.3083248 6.42727 7.63773   1  38   3   5  9.00
## 5%  7.9500 8.286989 400 0.3083248 7.34477 8.55523   1  69   3   5 10.25
##
## $comparison
## NULL
##
## $groups
##      value groups
## 5%  7.9500      a
## 10% 7.0325      b
```

```
##
## attr(,"class")
## [1] "group"
```

```
# Assumption Tests
## Residuals vs Fitted
plot(model, which=1)
```




```
## Q-Q Residuals
plot(model, which=2)
```



Conclusion

Validation

The experiment was statistically robust. The sample size per group was sufficient, and power analysis results confirm the reliability of the findings.

Analysis and Recommendation

The three-way ANOVA examined the effects of **Stack Algorithm**, **Ground Algorithm**, and **Corruption Rate** on the total number of moves required to solve the Tower of Hanoi puzzle with illegal states.

Key findings: * Evaluate which main effects and interactions are statistically significant from the ANOVA table * Identify which algorithm combinations minimize total moves * Assess how corruption rate impacts performance across different algorithm pairs * Use Tukey's HSD results to determine which specific combinations differ significantly

Recommendation: Based on the analysis results, select the algorithm combination that minimizes total moves while maintaining robustness across corruption rates.